

Megatron Core MoE 技术报告全文翻译（第五部分）

第7节 生产特性

前几节聚焦于突破三堵墙的性能优化。然而，生产级MoE训练还需要鲁棒性、灵活性和易用性。本节介绍满足这些运营需求的特性：保证稳定训练的负载均衡、共享专家与潜在专家架构、支持灵活部署的分布式checkpoint、利用现有稠密模型的upcycling，以及与多token预测等高级训练范式的集成。

7.1 负载均衡与Token丢弃

MoE模型的动态路由可能导致严重的工作负载不均衡——某些专家接收的token远多于其他专家，造成计算低效、内存瓶颈和硬件利用率下降。我们通过两种协调机制解决：负载均衡和token丢弃。

负载均衡。 图39展示了Megatron-Core MoE支持的负载均衡策略。我们提供多种策略来鼓励均匀的token分布：

辅助损失（Auxiliary Loss）： 一个可微惩罚项，阻止所有token路由到少数专家。

专家选择路由（Expert Choice Routing）： 将均衡路由表述为最优传输问题。

无辅助损失均衡： 通过可学习的专家偏置项动态调整路由决策，基于历史负载进行自适应调整（Aux-Loss-Free）。

Token丢弃。 支持两种调度策略：

Dropless模式（默认）： 所有路由token不受容量限制地处理，最大化模型表达能力，代价是每专家工作负载可变。

可丢弃模式： 显式强制容量上限。当分配给专家的token超过其容量时，多余token被丢弃并通过残差连接绕过。这提供了可预测的内存边界，在路由器初始化不好的早期训练中很有用。

Pad-to-Max静态形状。 可丢弃模式还支持“pad-to-max”功能——将所有专家输入填充到相同容量上限。这将MoE固有的动态每专家token数转换为静态形状，从而启用CUDA Graphs（第4.3.6节）等需要固定张量维度的优化。

7.2 共享专家

部分MoE架构（DeepSeek-V2/V3、Qwen等）包含一个共享专家，无论路由如何都处理所有token，为所有token提供一致的基线容量。当启用`--moe-shared-expert-overlap`时，共享专家计算与all-to-all通信和路由专家计算并行运行，将其延迟隐藏在dispatch-compute-combine流水线之后。

7.3 潜在MoE（Latent MoE）

在标准MoE中，all2all通信以完整隐藏维度 d 分发token，每个专家由形状为 $R^{m \times d}$ 和 $R^{d \times m}$ 的权重矩阵参数化。**LatentMoE**通过在dispatch前插入共享下投影 $W_{\downarrow}$ （压缩到潜在维度 ℓ ）、在combine后插入上投影 $W_{\uparrow}$ 来同时降低两者的开销，其中 $\ell < d$ 。路由仍在完整隐藏维度上进行以保证路由质量；每个路由专家完全在压缩的潜在空间中运行；共享专家保持维度 d 。

压缩比 $\alpha = d/\ell$ 将all2all通信量减少 α 倍（token以维度 ℓ 而非 d 分发），专家权重大小也相同比例缩小。这一架构已被NVIDIA的Nemotron-3 Super和Ultra模型采用。

实现： 通过`--moe-latent-size`启用，设置 ℓ 。MoELayer实例化两个TELinear投影：fc1_latent_proj（预处理步骤中的下投影，路由后、dispatch前）和fc2_latent_proj（后处理步骤中的上投影，combine后）。

7.4 分布式Checkpoint

传统checkpoint将保存的模型状态与特定并行配置紧密耦合，当改变TP、EP、PP或其他并行设置时需要复杂的离线转换。我们的分布式checkpoint库通过**并行无关的checkpoint保存与自动重分片**解决这一问题。

核心抽象是 `ShardedTensor` 描述符，它编码每个本地张量的全局形状、偏移量和分片模式。保存时，每个rank独立写入其本地分片（完全并行保存），消除协调者瓶颈。加载时，每个rank根据新的分片规格确定需要哪些全局张量的部分，并只读取那些切片。

这实现了任意到任意的并行配置切换：用TP=2、EP=4保存的checkpoint可以用TP=4、EP=8加载，无需离线转换。库支持Zarr（默认）和PyTorch Distributed存储后端。

7.5 灵活非对称虚拟流水线并行

交错流水线并行将每个物理流水线阶段划分为多个虚拟阶段，通过交错调度缓解流水线bubble。传统VPP需要均匀的层分布。然而，MoE模型存在显著的工作负载异质性——MoE层、稠密层、嵌入层、损失层和MTP等专用层具有显著不同的计算开销。

我们引入**灵活非对称VPP**，允许每个虚拟阶段有不同数量和类型的层。以PP=16、VPP=2的DeepSeek-V3（61个decoder层+1个MTP层）为例（见表10）：首个rank将轻量级嵌入与3个稠密decoder组合（匹配2个MoE层的开销）；大多数rank每阶段持有2个MoE decoder；最后几个rank策略性地放置沉重的MTP层和轻量级损失层以平衡工作负载。

这种方法实现了：(1)任意层数模型的VPP；(2)通过对每层计算开销的细粒度控制实现真正的负载均衡；(3)为MTP、编解码器结构等特殊层提供灵活的放置策略。

7.6 Upcycling

Upcycling将预训练的稠密模型转换为稀疏MoE架构，在不从头重新训练的情况下扩展模型容量。我们支持虚拟组初始化和专家权重缩放，以实现稠密checkpoint到细粒度MoE模型的无缝适配。该方法采用softmax-then-topK路由，在不变或降低推理成本的情况下提高表达能力。

具体做法：(1)在中间维度上切分MLP权重（4h → 2h）然后复制；(2)初始化一半路由器权重后复制，确保Top-2始终选择每个MLP分片之一，使MoE输出在训练初始时与稠密模型相同。

7.7 多Token预测（Multi-Token Prediction, MTP）

多token预测在每个位置优化模型以预测多个连续的未来token，增密监督信号。与并行独立预测不同，MTP通过隐藏状态转移维护预测之间的因果依赖，加速收敛并改善生成质量。推理时，模型恢复为标准的单token预测以保持部署兼容性。

我们将MTP与灵活流水线并行集成，允许MTP层在VPP布局中策略性地放置以实现均衡的工作负载分布。

7.8 Muon优化器

与AdamW等逐元素更新的传统优化器不同，Muon优化器通过对整个权重矩阵进行正交化引入了一种矩阵感知方法。这改善了优化轨迹的条件数，与AdamW相比显著减少了训练步数。

我们的集成为大规模分布式训练提供了生产就绪的支持：(1)完整支持分割QKV权重布局，即使注意力投影矩阵以独立张量存储也能高效正交化；(2)与分布式优化器无缝集成，允许跨数据并行rank分片优化器状态同时保持正确的正交化语义；(3)当GPU内存受限时，对Muon的正交化缓冲区进行CPU卸载。

MuonClip。万亿参数模型训练引入了稳定性挑战——query-key点积可能无限增长，导致注意力爆炸。

MuonClip通过cuDNN、cudnn-frontend和Transformer Engine中的硬件加速实现解决这一问题。

第8节 性能评估

本节通过实证评估验证前述贡献的有效性，展示Megatron-Core MoE在多种模型配置和硬件平台上的性能。

注意（Living Document）：本节性能结果是基于Megatron-Core v0.16的时间点快照，并非可达性能的上限。持续的优化工作正在进行中，这些数字将在未来版本中改进。我们也计划随着社区新模型架构的出现扩展覆盖范

围。最新性能数据请参阅Megatron-LM和Megatron-Bridge代码仓库。

8.1 实验设置

基准模型。 我们在两种最先进的细粒度MoE架构上评估：DeepSeek-V3-685B和Qwen3-235B。两个模型都具有放大三堵墙挑战的细粒度专家设计：高专家数量给内存容量带来压力，top-k路由增加all-to-all通信量，更小的per-expert计算降低GEMM效率。

硬件与软件栈。 基准测试在NVIDIA GB200和H100 GPU上进行。我们使用Megatron-LM的dev分支配合最新的TransformerEngine。启用第4节描述的完整优化栈，包括但不限于：

FP8训练： GB200/GB300（Blackwell）上使用原生Tensor Core支持的MXFP8，H100（Hopper）上使用blockwise FP8

内存优化： 选择性重计算和激活/优化器状态卸载

优化token dispatcher： GB200/GB300 NVL系统上利用多节点NVLink的HybridEP，H100系统上的DeepEP

通信重叠： 1F1B all-to-all与专家计算的重叠

Kernel优化： 分组GEMM、路由融合、置换融合和CUDA Graphs

指标。 我们报告每GPU的TFLOPS吞吐量和每GPU每秒处理的token数。这两个指标提供了计算效率和实际训练吞吐量的两种视角。

8.2 关键性能结果

总览。 表11汇总了在NVIDIA GB300、GB200和H100系统上，标准序列长度4096下两种细粒度MoE训练工作负载在完整优化栈下的每GPU吞吐量。所有配置使用强制均衡路由以确保跨专家的均匀token分布。

表11：两种MoE模型在NVIDIA GB300、GB200和H100上的统一吞吐量基准（每GPU数据）。所有配置使用强制均衡路由。Dtype列指定FP8配方：FP8-BLK表示Hopper上的blockwise FP8，MXFP8表示Blackwell上的微缩放FP8。

模型	系统	GPU数	序列长	Dtype	每GPU TF	Tokens/s/GPU
DeepSeek-V3	GB300	256	4096	MXFP8	1233	4730
DeepSeek-V3	GB200	256	4096	MXFP8	1048	4020
DeepSeek-V3	GB200	256	4096	BF16	857	3298
DeepSeek-V3	H100	1024	4096	FP8-BLK	368	1412
Qwen3-235B	GB300	256	4096	MXFP8	974	6583
Qwen3-235B	GB200	256	4096	MXFP8	919	6212
Qwen3-235B	GB200	256	4096	BF16	750	5100
Qwen3-235B	H100	256	4096	BF16	320	2132
Qwen3-235B	GB300	128	131072	MXFP8	1150	1556

FP8训练有效性。 FP8训练在H100上为DeepSeek-V3实现了368 TFLOPS/GPU，表明blockwise FP8配方能够高效训练Hopper架构上的大规模MoE模型，在保持数值稳定性的同时提供低精度的性能收益。

GB200和GB300性能。 GB200和GB300平台相比H100在相当或更少GPU数量下，两种MoE模型的token吞吐量均提升约3倍。这一效率提升源于更高的内存带宽、更强的计算能力和原生MXFP8 Tensor Core支持。结合软件优化，GB200在DeepSeek-V3训练中超过1048 TFLOPS/GPU，GB300进一步达到1233 TFLOPS/GPU。

可扩展性。 结果表明Megatron-Core MoE能高效扩展到具有数千GPU和数千亿参数的生产工作负载。DeepSeek-V3在1024 GPU规模的成功部署验证了并行折叠（第3.3节）、优化dispatcher（第4.2节）和kernel优化（第4.3节）的有效性。

长上下文压力测试。 为捕获超出标准4096 token区间的行为，表11包含了在GB300上、序列长度131072的Qwen3-235B长上下文运行。即使在这种内存和通信密集的设置下，系统仍维持1150 TFLOPS/GPU，表明完整优化栈下强大的长上下文效率。

第9节 性能最佳实践

第3–7节提供了丰富的MoE训练优化手段：并行折叠、优化dispatcher、分组GEMM、低精度训练、CUDA Graphs等。然而，拥有众多优化技术并不能自动转化为高性能。每种技术解决特定瓶颈，但引入其自身的权衡。挑战不在于优化技术的可用性，而在于知道**应用哪些、何时应用，以及它们如何相互作用**。

本节第9.1节提供系统性方法论，第9.2节通过DeepSeek-V3案例展示这些原则的组合应用。

9.1 系统性优化 workflow

我们提出三阶段 workflow，通过优化Mixtral、DeepSeek-V3和Qwen3在GB200和H100上的经验总结而来。这一过程本质上是迭代的。

9.1.1 阶段一：建立内存可行的并行配置

内存可行性是首要约束。在优化吞吐量之前，配置必须适配GPU内存。表12总结各并行策略对每GPU内存和通信开销的影响。

表12：并行策略对内存和通信的影响（d = 并行度）

策略	峰值激活	权重内存	优化器状态	通信（每层）
TP	1/d（含SP）	1/d	1/d	高
EP	~1（负载依赖）	1/d（仅MoE）	1/d	中
PP	1（VPP>1则更低）	1/d	1/d	中
CP	1/d	1	1/d ⁺	中
DP	1	1	1/d ⁺	低

⁺需要分布式优化器（`--use-distributed-optimizer`）

快速可行性测试： 使用`--fake-init-process-group`在单GPU上模拟分布式训练，无需分配完整集群即可快速迭代并行配置。

9.1.2 阶段二：选择最优并行策略

确定内存可行配置后，选择在保持吞吐量的同时最小化通信开销的策略。最优选择取决于模型架构、序列长度和硬件拓扑。

指南1：最小化模型并行度，最大化数据并行度

目标：在不OOM的情况下，保持TP/EP/PP/CP尽可能小。

原因：模型并行引入损害性能的通信开销。

方法：使用分布式优化器（`--use-distributed-optimizer`）跨DP rank分片优化器状态，释放内存用于更大的DP。

指南2：将EP和TP通信保持在NVLink域内

目标：确保EP×TP在NVLink域内（通常单节点8 GPU，除非使用多节点NVLink）。

原因：EP和TP通信密集，NVLink提供远高于跨节点互联的带宽。

扩展：当扩展超出NVLink域时，优先使用PP而非跨节点扩展TP/EP。

指南3：使用PP进行多节点扩展

目标：使用PP跨节点分布层，同时保持EP×TP在NVLink内。

VPP：当PP≥2时启用虚拟流水线并行以减少流水线bubble。

配置：设置`--pipeline-model-parallel-layout`控制PP设置（见7.5节）。较大VPP通常减少bubble但增加P2P通信，中间值通常提供最佳平衡。

指南4：对专家层优先选EP而非TP

EP相比TP对MoE层有几个优势：

更好的GEMM效率：更大的本地矩阵尺寸提高GPU利用率。

更低的通信：MoE层的EP通信开销比TP更低。

更简单的计算图：更容易将通信与计算重叠。

Token置换：当EP = num_experts时，消除本地token置换。

示例：对于Mixtral-8×7B，EP8×TP1优于EP4×TP2。

指南5：长序列启用上下文并行（CP）

使用时机：序列长度 ≥ 8K token。

避免时机：序列 < 4K token时，CP开销通常超过收益。

关键因素：CP效率取决于通信与计算的重叠。

9.1.3 阶段三：分析并优化瓶颈

建立有效的并行配置后，分析训练运行以确定哪堵墙占主导，然后应用针对性优化。

内存瓶颈（内存墙）

症状：被迫使用完整重计算或过大的并行度以避免OOM。

解决方案（表13）：

优化	开销	配置
FP8训练	低	<code>--fp8-format --fp8-recipe</code>
选择性重计算	低	<code>--recompute-granularity --recompute-modules</code>
精度感知优化器	低	<code>--use-precision-aware-optimizer</code>
激活卸载	中	<code>--fine-grained-activation-offloading --offload-modules</code>
优化器卸载	中	<code>--offload-optimizer-states</code>

通信瓶颈（通信墙）

症状：分析显示collective操作中花费大量时间。

解决方案（表14）：

通信类型	配置
DP梯度规约和参数gather	<code>--overlap-grad-reduce --overlap-param-gather</code>
TP通信	<code>--tp-comm-overlap</code>
EP dispatcher	<code>--moe-token-dispatcher-type</code>
EP all-to-all隐藏	<code>--overlap-moe-expert-parallel-comm</code>
PP发送/接收	<code>--pipeline-model-parallel-layout</code>

CPU开销瓶颈（计算效率墙）

症状：Nsight Systems时间线显示GPU kernel之间存在间隙，CPU无法足够快地启动kernel。

解决方案（表15）：

优化	配置
禁用Python GC	<code>--manual-gc --manual-gc-interval 10</code>

减少kernel启动次数 降低TP或增大MBS

启用CUDA Graphs `--cuda-graph-impl transformer_engine`

计算低效瓶颈（计算效率墙）

症状：尽管没有通信或CPU瓶颈，GPU SM利用率仍然很低。

解决方案（表16）：

优化	配置
分组GEMM	<code>--moe-grouped-gemm</code>

Kernel融合 `--moe-router-fusion --moe-permute-fusion`

FP8精度 `--fp8-format --fp8-recipe`

典型对比示例： 在NVL8系统上，EP跨节点，分析可能显示all-to-all通信占30-50%的步骤时间，直指通信墙。在NVL72系统上，相同EP度保持在NVLink域内，启用FP8后主要瓶颈往往转移到CPU开销——更快的GPU计算暴露了host端启动延迟。**同一模型在不同硬件上可能需要完全不同的优化策略。**

9.1.4 小结

三个阶段的顺序至关重要：内存约束是必须首先解决的硬性障碍（阶段一），并行决策决定通信拓扑（阶段二），只有这样分析才能揭示要攻克哪堵墙（阶段三）。这一过程本质上是迭代的——内存优化可能启用更小的并行度，需要返回阶段一。某些阶段三优化有其自身内存开销（EP通信重叠需要额外缓冲，CUDA Graphs消耗额外内存），可能需要重新审视早期决策。每轮变更后持续分析以指导下一轮迭代。

9.2 案例研究：在GB200和H100上调优DeepSeek-V3

DeepSeek-V3代表了MoE训练的极端压力测试：685B总参数、多token预测（MTP）和多潜在注意力（MLA）。三堵墙全面压力：256个专家的内存压力、top-8路由的高all-to-all通信量、小的per-expert GEMM。

9.2.1 最终优化配置与性能

表17： DeepSeek-V3在GB200和H100上的最终优化配置（+使用并行折叠； TP仅适用于非MoE模块， 专家TP始终为1）

配置项	GB200		H100
硬件	256×GB200		1024×H100
并行（TP/PP/EP） +	1/4/64	2/8/64	
VPP	4	4	
GBS/MBS/SeqLen	8192/1/4096		8192/1/4096
精度	MXFP8	FP8-Blockwise	
Dispatcher	HybridEP	DeepEP	
重计算	mlp	mlp, mla_up_proj, moe_act, layernorm	
CUDA Graphs	启用	—	
EP all-to-all重叠	—	启用	
性能（TFLOPS/GPU）	1048	368	

GB200配置使用HybridEP（针对NVL72优化）和CUDA Graphs来最小化CPU开销——这在Blackwell的FP8训练中成为主要瓶颈。H100配置使用FP8-blockwise精度配合DeepEP和EP all-to-all重叠来隐藏通信延迟。

9.2.2 优化配置解析

为何选择这种并行布局？

两个平台均使用并行折叠（指南4）将注意力和专家并行解耦：TP仅用于注意力层以节省内存，专家使用TP=1的EP以获得更好的GEMM效率。EP64使每GPU恰好持有256个专家中的4个，消除本地token置换开销。

关键的平台差异在于内存容量：GB200的192 GB/GPU（相比H100的80 GB）允许TP1/PP4而非TP2/PP8，将流水线深度减半，减少流水线bubble并简化工作负载均衡。灵活VPP对流水线进行微调：H100布局[Et*3]

(ttl)*29m|L]将嵌入与3个transformer层分在首阶段，将MTP单独一阶段，损失单独一层；GB200布局[Et*4](ttttl)*14tmL]在更短的流水线上跨16个虚拟阶段分布层。

内存墙

FP8训练是第一个杠杆：在两个平台上将激活内存减半，释放GPU内存。在H100上，释放的内存预算至关重要，因为它支持EP通信重叠（需要流水线dispatch和combine操作的额外缓冲，见下文通信墙）。细粒度重计算（mlp、mla_up_proj、layernorm、moe_act）削减剩余内存压力。内存高效置换消除冗余激活存储，低精度优化器状态（BF16）提供额外余量。

在GB200上，内存链的解决方式不同。NVL72将EP保持在本地，通信重叠不是关键，其内存预算完全释放。GB200更高的C2C带宽（NVLink-C2C）使优化器状态卸载有效：传输开销足够低，卸载释放大量GPU内存用于激活，从而减少细粒度重计算的需求，仅需重计算mlp。

通信墙

在H100/B200（NVL8）上，EP64跨8个节点，未经优化的跨节点all-to-all延迟将消耗约50%的步骤时间。DeepEP通过融合置换和优化collective操作降低这一开销。即便如此，EP通信重叠对于将剩余跨节点延迟隐藏在专家计算后面仍然至关重要，而这只有在FP8释放了足够内存用于额外缓冲之后才可行（见内存墙以上内容）。

在GB200（NVL72）上，EP64完全在NVLink域内。HybridEP充分利用1.8 TB/s双向带宽，无需通信重叠。通信墙实际上仅靠硬件拓扑就解决了，将主要瓶颈转移到计算效率。

计算效率墙

FP8在两个平台上加速GEMM（H100使用blockwise FP8，GB200使用原生Blackwell Tensor Core支持的MXFP8），但这一加速有副作用：更快的GPU计算暴露了CPU开销。在GB200上，NVL72已经消除了通信瓶颈，CPU开销成为主要约束——host端无法足够快地启动kernel以保持GPU饱和。

部分CUDA Graphs通过将注意力、路由器和MoE预处理捕获到静态图中来解决这个问题，同时让动态专家计算保持ungraphed状态。Kernel融合（路由融合、置换融合、MLA RoPE融合）减少kernel启动次数。

CPU/NUMA绑定降低host端内存访问延迟。

9.2.3 经验教训

四条可推广的洞见：

平台特性驱动策略：GB200更大的内存（192GB vs 80GB）和更高的C2C带宽支持更激进的选择：更小的PP度、更少的细粒度重计算和有效的优化器卸载。H100的NVL8需要EP通信重叠来隐藏跨节点延迟，而GB200的NVL72将EP64保持在NVLink域内。

并行折叠释放灵活性：将注意力TP与专家EP解耦允许各层类型独立优化。配合灵活VPP，这实现了跨流水线阶段的细粒度工作负载均衡。

FP8转移瓶颈：FP8训练加速GEMM并减少内存，但放大CPU开销为主要瓶颈。CUDA Graphs、kernel融合和CPU/NUMA绑定变得至关重要。

迭代优化：优化过程本质上是循环的。内存优化（重计算、卸载）释放GPU内存，支持通信重叠。通信优化暴露计算效率瓶颈。某些优化有跨领域效果：FP8训练同时减少内存和计算时间，但增加CPU开销；CUDA Graphs减少CPU开销，但消耗额外内存。这些相互作用意味着应用一个优化可能需要重新审视早期决策。每次变更后持续分析，识别收益递减的时机以转向下一个瓶颈。

第10节 Megatron-Core MoE在强化学习中的应用

强化学习（RL）后训练在OpenAI o1和DeepSeek-R1之后已成为关键范式。许多领先的RL训练模型是MoE架构（DeepSeek-R1、Kimi-K2等），而RL工作负载引入了放大本报告中MoE特有问题的挑战：高度可变的序列长度给内存墙带来压力，推理和训练阶段交替进行要求快速内存卸载，推理和训练引擎之间的路由差异威胁训练稳定性。

10.1 RL后训练的挑战

RL后训练与预训练的差异在几个方面对训练引擎提出了新要求：

变长序列。预训练通常在固定长度序列（如4K token）上运行。RL工作负载产生高度可变的序列长度：最大值可达128K甚至1M token，而批内均值通常是最大值的二分之一到四分之一。这种长尾分布使得难以平衡计算效率与峰值内存消耗。

内存卸载。RL框架通常在同一GPU上共置训练引擎和推理引擎，在一方活跃时卸载另一方的状态。这要求两个引擎都能快速完整地释放和恢复其内存占用——当优化器状态、激活和KV cache都需要管理时，这是个非平凡的要求。

在线权重导出。每次训练步骤后，推理引擎必须以推理引擎可以加载的格式快速接收最新参数，跨越多样化的模型架构。

训练稳定性。标准RL算法假设采样响应来自当前策略分布。然而实际上，推理和训练引擎使用不同的优化kernel，即使参数完全相同也会产生略微不同的token概率——实际上引入了离策略偏差。对于MoE模型，这个问题更加复杂：同一序列中的token可能在推理和训练引擎中被路由到不同的专家，放大了差异。

10.2 Megatron-Bridge

典型的RL workflow从预训练的HuggingFace模型开始，并在RL框架中进行微调。在规模上，这些框架通常在Ray worker内运行Megatron-Core进行分布式训练，而rollout通常使用期望HuggingFace格式checkpoint的推理栈。这产生了两个集成需求：将模型定义映射到Megatron-Core模块，以及在HuggingFace和Megatron格式之间高效转换checkpoint。

Megatron-Bridge解决了checkpoint互操作性层，支持用于初始化、训练和导出的快速HF到Megatron转换。这一模式在多个RL框架中使用，包括veRL、Slime和NeMo RL。

10.3 面向强化学习的Megatron-Core优化

序列打包支持。如第6.4.1节所述，打包序列消除了变长序列批次中的填充浪费。基于序列打包支持，在RL框架侧，我们推荐使用**打包感知的动态批大小**，确保每个批次有相似的有效token总数。此外，我们引入了一种额外的负载均衡策略，显式考虑transformer块的异构计算成本——注意力模块随序列长度二次方扩展 ($O(L^2)$)，而FFN线性扩展 ($O(L)$)，token数均衡的批次在wall-clock时间上可能仍然高度不均衡。

为缓解这一问题，我们计算批内每个样本序列长度的平方并求和作为“注意力成本”指标，然后按此排序微批并以小→大→小的蛇形模式调度（升序后降序）。这种蛇形排序带来两个好处：(i)对DP、PP和EP，减少同步bubble；(ii)在PP的预热和冷却阶段，较轻的微批先到来，减少空闲时间。

动态上下文并行。如第6.4.2节所述，传统训练需要选择一个固定的CP度以保证最长序列不OOM，这迫使大多数较短序列（内存占用远小于最长序列）也使用不必要大的CP度，浪费跨设备带宽并降低总体吞吐量。动态CP通过逐微批自适应选择CP度消除了这种一刀切的限制，将到来序列按长度分桶，为每个桶选择在设备内存预算内的最小CP度。

CPU优化器卸载。为进一步缓解GPU内存压力，在前向和反向pass期间将优化器状态卸载到host DRAM，仅在参数更新步骤时将其加载回GPU。由于优化器张量可能占模型参数的数倍大小，临时驱逐它们可以释放大量高带宽GPU内存用于缓存激活或容纳更长序列。

FP16训练支持。尽管BF16已被广泛用于大模型预训练，一些研究观察到在RL训练中，半精度浮点（FP16）在某些超参数选择下可以提供更好的数值稳定性。Megatron-Core MoE因此提供完整功能的FP16路径，包括损失缩放和混合精度优化器kernel，使从业者无需牺牲吞吐量即可选择最符合其稳定性要求的精度模式。

路由器重放（Router Replay）。最新研究表明，捕获MoE路由器在推理期间产生的路由决策并在后续训练阶段重放，可以改善收敛一致性。得益于社区贡献的补丁，Megatron-Core MoE现在支持这一特性：推理引擎记录每个token的专家分配，训练栈可以读取并强制执行相同的路由模式。这一机制将路由可变性与权重更新解耦，带来更稳定的优化轨迹，在策略分布频繁变化的RL场景中尤为有效。

第11节 结论

本报告介绍了Megatron-Core MoE，一个用于训练大规模混合专家模型的开源框架。MoE稀疏性引入了两个根本性挑战：**参数-计算失配**（表现为三堵墙：内存墙、通信墙和计算效率墙）和**稠密-稀疏失配**（需要为注意力和MoE层解耦并行配置）。通过对这些挑战的集成解决方案，Megatron-Core MoE实现了万亿参数级别的高效训练。

关键技术贡献：

多维并行与并行折叠：EP与TP、PP、CP、DP无缝集成；MoE并行折叠打破 $EP \leq DP$ 约束，实现针对模型架构和硬件拓扑的灵活并行映射。

内存优化：细粒度激活重计算、内存高效置换、BF16动量的精度感知优化器和CPU激活卸载，将DeepSeek-V3的每GPU内存需求从199.5 GB降至可行范围，使当前硬件上的万亿参数训练成为可能。

通信优化：高性能token dispatcher（DeepEP和HybridEP）以及通信-计算重叠，将all-to-all从瓶颈转变为隐藏在专家计算后面的后台操作，将开销从高达60%的训练时间降至不足10%。

计算效率：分组GEMM kernel批量处理专家计算以提高硬件利用率，kernel融合整合路由和置换操作，CUDA Graphs减少启动开销，sync-free执行消除dropless MoE的host-device同步。

FP8训练：完整的FP8支持配合选择性精度——保护数值敏感组件（路由器、嵌入层、优化器状态）同时激进量化批量计算（专家GEMM）——同时影响三堵墙。

长上下文训练：CP和TP扩展维持每设备恒定的子序列长度，支持16K到64K+的token序列训练，注意力计算主导时同样高效。

生产特性：负载均衡和token丢弃保证稳定训练，分布式checkpoint支持并行无关的重分片，upcycling从稠密checkpoint初始化MoE模型，MTP集成支持高级训练目标。

强化学习支持：Megatron Core和Megatron-Bridge与主流RL框架无缝集成，序列打包、动态上下文并行和路由器重放解决RL后训练工作负载的独特挑战。

这些优化使Megatron-Core MoE实现了强劲的训练吞吐量：DeepSeek-V3（685B参数，256个专家）在256块GB300/GB200上分别达到1233/1048 TFLOPS/GPU，在1024块H100上达到368 TFLOPS/GPU；Qwen3-235B在GB300/GB200上达到974/919 TFLOPS，在H100上达到320 TFLOPS。GB300和GB200平台相比H100提供约3×的token吞吐量，展示了框架利用下一代硬件的能力。

通过开源这些能力，Megatron-Core MoE为研究人员和从业者提供了在规模上进行MoE实验和部署的生产级工具。模块化架构支持快速原型设计，完整优化栈支持从研究原型到万亿参数生产模型的训练。